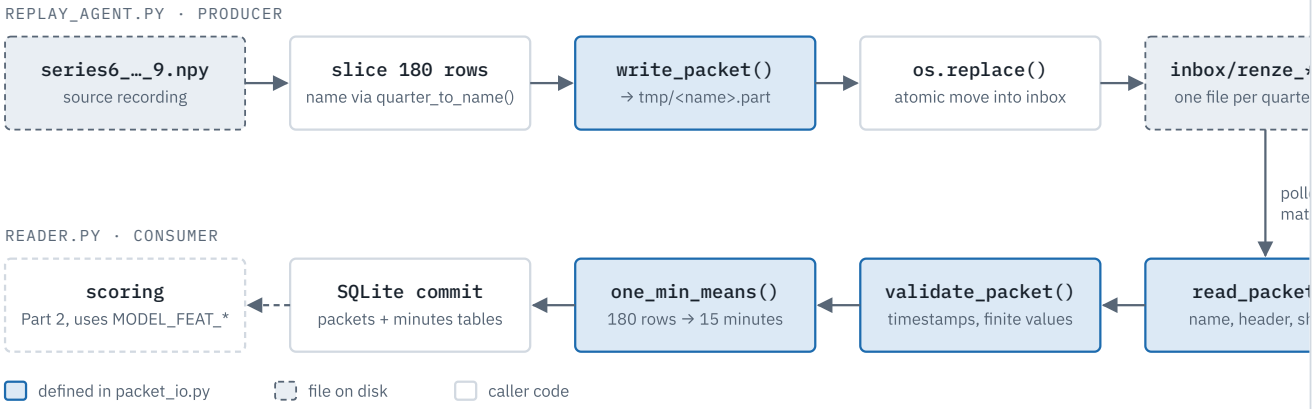
constants and functions, the file format can only change in one place.

5 s sampling interval per row	**180 × 50** rows × columns in every packet (15 min)	**49 → 48** raw sensors, PT104S dropped	**11** sensors the model uses
15 × 48 one-minute means per packet			

1 Where it sits in the pipeline

Highlighted boxes are functions defined in `packet_io.py`. Everything else belongs to `replay_agent.py` or `reader.py`. The only thing the two processes share is the inbox folder, so the file format is their entire interface.



Writing to `tmp/….part` and then renaming means the reader never sees a half-written file. The `.part` name also fails the filename regex, so even a stray partial file would be ignored.

2 The file name

Each packet is named after the UTC start of its quarter-hour. `quarter_to_name()` turns an epoch timestamp into this name and `name_to_quarter()` turns it back, returning `None` for anything that doesn't match `^renze_(\d{8}T\d{4}Z)\.csv$`.

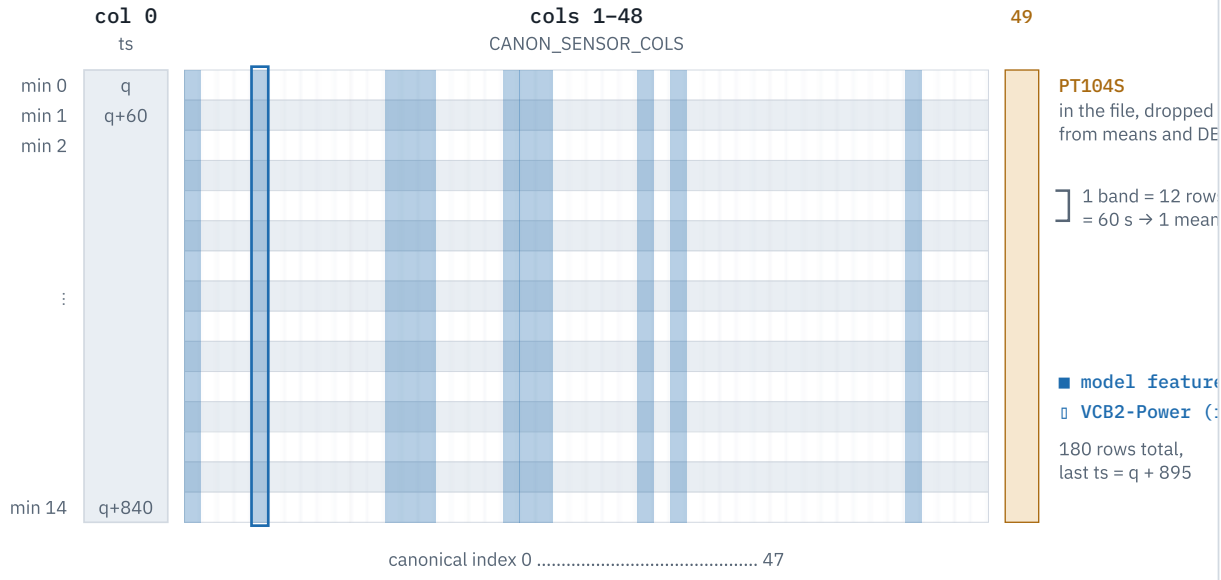
```
renze_20240315T0915Z.csv
```

prefix · YYYYMMDD · sep · HHMM · UTC · ext

```
# round trip
quarter_to_name(1710494100)            # → 'renze_20240315T0915Z.csv'
name_to_quarter('renze_20240315T0915Z.csv')  # → 1710494100
name_to_quarter('renze_20240315T0915Z.csv.part')  # → None (ignored)
```

3 Inside a packet

A packet CSV has a header line (`ts,` followed by the 49 raw sensor names) and exactly 180 data rows. Column 0 is the epoch timestamp, which goes up by 5 seconds per row. Columns 1–48 are the sensors the rest of the system uses. Column 49, `PT104S`, is carried in the file but dropped when means are computed.



q is the quarter's epoch start taken from the file name. The ts column holds every 5 s value from q to q+895; the labels show the first row of each one-minute band.

4 Three column lists

The module defines the sensor columns at three levels of selection. Each one is derived from the one above it, so the order is fixed by `RAW_SENSOR_COLS`.

`RAW_SENSOR_COLS`
what the CSV holds
49

`CANON_SENSOR_COLS`
RAW[:-1], used by reader and SQLite
48

`MODEL_FEAT_COLS`
picked by MODEL_FEAT_INDICES
11

CANON IDX	CSV COL	SENSOR	TAG TYPE (ISA LETTERS)
0	1	**VCB1-Power**	Breaker 1 power
4	5	**VCB2-Power**	Breaker 2 power, also `VCB2_IDX`
12	13	**PT100**	Pressure transmitter
13	14	**PT101**	Pressure transmitter
14	15	**PV101**	Pressure valve
19	20	**LIT101**	Level indicating transmitter
20	21	**LV101**	Level valve
21	22	**TT101**	Temperature transmitter
27	28	**FT102**	Flow transmitter
29	30	**FT103**	Flow transmitter
43	44	**ENV-0**	Environmental reading

Tag descriptions are read from standard instrument-tag letters (P/T/L/F + T/V), not from the code. The CSV column is always the canonical index + 1, because column 0 is the timestamp. `replay_agent.py` relies on this when it computes `VCB2_IDX + 1`.

VCB1–3 power, voltage, current, PF ×3 | **PT** pressure | **TT** temperature | **LIT** level | **FT** flow | **PV / LV / TV** valves | **VFDB / VFDC** drives | **ENV-0** environment

5 The six functions

quarter_to_name

`(quarter_ts: int) → str`

Formats an epoch-second timestamp as `renze_YYYYMMDDTHHMMZ.csv` in UTC.

used by: replay_agent, reader

name_to_quarter

`(name: str) → int | None`

Parses a file name back to its epoch timestamp. Returns `None` instead of raising, so callers can use it as a filter when scanning the inbox.

used by: replay_agent, reader

write_packet

`(path, rows: ndarray[180,50]) → None`

Writes the header and rows with `np.savetxt`. Timestamps are written as integers (`%d`), sensor values with 9 significant digits (`%.9g`).

used by: replay_agent

read_packet

`(path) → (quarter_ts, ndarray[180,50])`

Checks the file name, then the header line, then loads the numbers and checks the shape. Raises `ValueError` at the first problem.

used by: reader

validate_packet

`(quarter_ts, data) → None`

Checks the content: timestamps must be exactly q, q+5, …, q+895, and every sensor value must be finite (no NaN or ±inf).

used by: reader

one_min_means

`(data) → (min_ts[15], means[15,48])`

Drops `ts` and `PT104S`, averages each block of 12 rows, and returns the start time of each minute.

used by: reader

6 What the reader checks, in order

The reader calls `read_packet` and then `validate_packet`. A packet must pass every step below before any of it reaches the database.

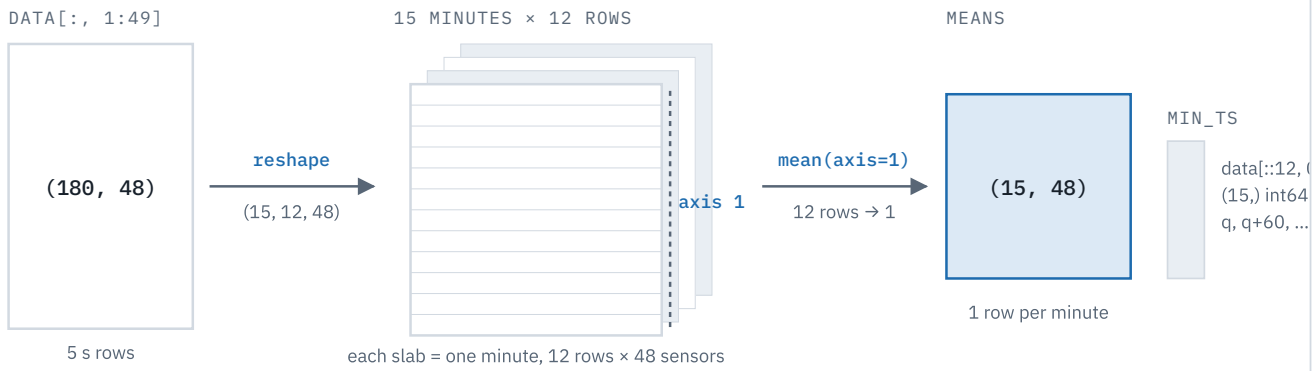
1	READ_PACKET · NAME File name matches the regex and parses to a UTC timestamp.	ValueError
2	READ_PACKET · HEADER First line equals `ts,VCB1-Power,…,PT104S` exactly, so column order is enforced.	ValueError
3	READ_PACKET · SHAPE	ValueError

`np.loadtxt` result is 2-D and exactly 180 × 50. Short, long, or ragged files fail here.

4	VALIDATE_PACKET · TIME The ts column equals `arange(q, q+900, 5)`, so there are no gaps, duplicates, or shifts, and the data agrees with the file name.	ValueError
5	VALIDATE_PACKET · VALUES All 49 sensor columns are finite. A `nan` or `inf` anywhere rejects the whole packet.	ValueError

7 How one-minute means are computed

There is no loop. The function reshapes the array so that each minute's 12 rows sit on their own axis, then averages along that axis.




```
canon    = data[:, 1:49]                # drop ts (col 0) and PT104S (col 49)
reshaped = canon.reshape(15, 12, 48)    # minute, row-in-minute, sensor
means    = reshaped.mean(axis=1)        # (15, 48)
min_ts   = data[::12, 0].astype(int64)  # rows 0, 12, 24, … 168
```

The reader then stores each of the 15 rows in SQLite, and uses `means[i, VCB2_IDX]` to update a rolling 15-minute VCB2-Power value.

8 Things to know

WATCH

Quarter alignment isn't enforced

`name_to_quarter` accepts any minute, so `renze_20240315T0907Z.csv` passes. `validate_packet` only checks that the timestamps agree with the name. If

packets must start on :00, :15, :30, or :45, add a check that `quarter_ts % 900 == 0`.

WATCH

The 1:49 slice is hard-coded

`one_min_means` assumes `PT104S` is the last column and that `validate_packet` has already run. If a sensor is ever added to `RAW_SENSOR_COLS`, this slice and the `%.9g` × 49 format in `write_packet` need to change too. Deriving both from `len(…)` would remove that risk.

INFO

CSV values are rounded to 9 significant digits

Writing with `%.9g` means values read back are not bit-identical to the source `.npy`. This is far below sensor noise, but exact-equality comparisons against the source will fail.

INFO

Model feature lists are not used yet

`MODEL_FEAT_INDICES` and `MODEL_FEAT_COLS` aren't imported by `reader.py` or `replay_agent.py`. The comment says they apply to both the pipe and HXG models and that notebook 09-B's 16 names map to these same 11 indices. They're waiting for the scoring step.

DESIGN

Strict but cheap checks come first

`read_packet` rejects bad names and headers before parsing any numbers, so malformed files fail quickly. It does open the file twice (once for the header, once for `loadtxt`), which is harmless at one file per quarter-hour.